Tytuł: **Basys Tanks**

Autorzy: **Antoni Zasadni (AZ), Kacper Wałęga (KW)**

Ostatnia modyfikacja: 22.08.2025

1. Repozytorium git ... 1
2. Wstęp ... 1
3. Specyfikacja ... 2
   3.1. Opis ogólny algorytmu ... 2
   3.2. Tabela zdarzeń ... 2
4. Architektura ... 3
   4.1. Moduł: top ... 3
      4.1.1. Schemat blokowy ... 3
      4.1.2. Porty ... 3
         a) ps2 – ps2_keyboard - input, uart - top_uart - input, sw[1:0] - uart_top - input 3
         b) vga – top_ctl, output ... 4
      4.1.3. Interfejsy ... 4
         a) vga_if– vga_timing,bg,start_screen,main_game,end_screen,top_ctl ... 4
   4.2. Rozprowadzenie sygnału zegara ... 5
5. Implementacja ... 6
   5.1. Lista zignorowanych ostrzeżeń Vivado ... 6
   5.2. Wykorzystanie zasobów ... 6
   5.3. Marginesy czasowe ... 7
6. Konfiguracja sprzętu ... 7
7. Film ... 9
8. Niezaimplementowane funkcje ... 9

1. Repozytorium git

Adres repozytorium GITa:

https://github.com/ZasadniAntoni/Basys_Tanks_AZ_KW

W przypadku repozytorium prywatnego należy zaprosić użytkownika zewnętrznego o adresie mailowym: kaczmarczyk@agh.edu.pl

2. Wstęp

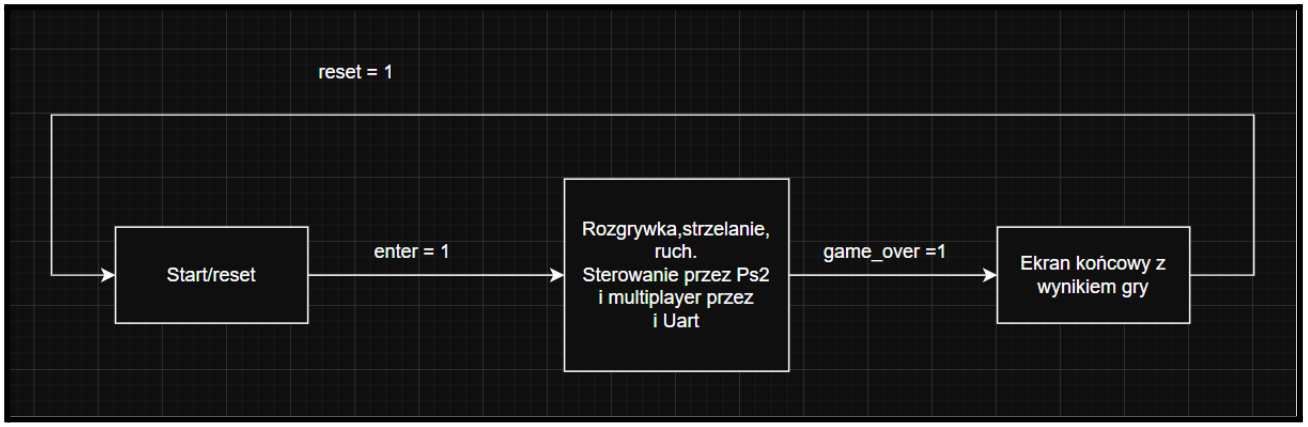
Pomysł wziął się z burzy mózgów na temat tego jaka oryginalna oraz kultowa gra w którą mogą grać 2 osoby jesteśmy w stanie zaimplementować. Dosyć szybko pomyśleliśmy o grze która bardzo przypomina kultowe Worms 2D, oraz jej pokrewnych tytułach. Finalnie stanęło na grze Pocket Tanks, która jest nieco prostsza, gdyż wymaga tylko 2 postacie zamiast kilku mniejszych robaków. Stąd też nasz tytuł jako próba odtworzenia tej prostej formuły na dobrą zabawę z rywalizacją. Przedstawiamy Basys Tanks.

3. Specyfikacja

3.1. Opis ogólny algorytmu

Uproszczony schemat blokowy działania implementowanego algorytmu. Co się dzieje po starcie, jak wygląda przebieg działania, kiedy i pod jakimi warunkami się kończy.

Ewentualnie przykładowe screen-shoty tego, co w przybliżeniu chcielibyśmy uzyskać.



3.2. Tabela zdarzeń

Opis zdarzeń występujących podczas działania programu/urządzenia, zarówno zewnętrznych (interakcje z użytkownikiem), jak i wewnętrznych (specyficzne stany w algorytmie). Zdarzenia podzielone są na kategorie dotyczące różnych stanów działania programu. Kategorie powinny odpowiadać stanom ze schematu z pkt. 2.1.

Zdarzenie	**Kategoria**	**Reakcja systemu**
Enter	Ekran Startowy	Gotowość Gracza
H	Gra	Otwarcie okna z helpem (opis, co robi który z wykorzystywanych inputów na klawiaturze)
Strzałka w prawo	Gra	Ruch czołgiem w prawo
Strzałka w lewo	Gra	Ruch czołgiem w lewo
Strzałka w górę/dół	Gra	Dostosowanie kąta lufy czołgu
Życie jednego z czołgów spada do zera	Gra	Koniec gry
Spacja (Przytrzymanie)	Gra	Zwiększanie się wartości odpowiadającej za prędkość i siłę pocisku
Spacja (Puszczenie)	Gra	Zapisanie wartości i strzał
Pocisk trafia w czołg przeciwnika	Gra	Przeciwnik traci punkt życia
ff15 (napisane w sekwencji)	Gra	Gracz się poddaje, grę zwycięża przeciwnik (łatwy sposób na przejście do end_screen, można szybko sprawdzić działanie internal_rst)
Enter	Ekran Końcowy	Reset wewnętrzny stanu gry, przejście do ekranu startowego

4. Architektura

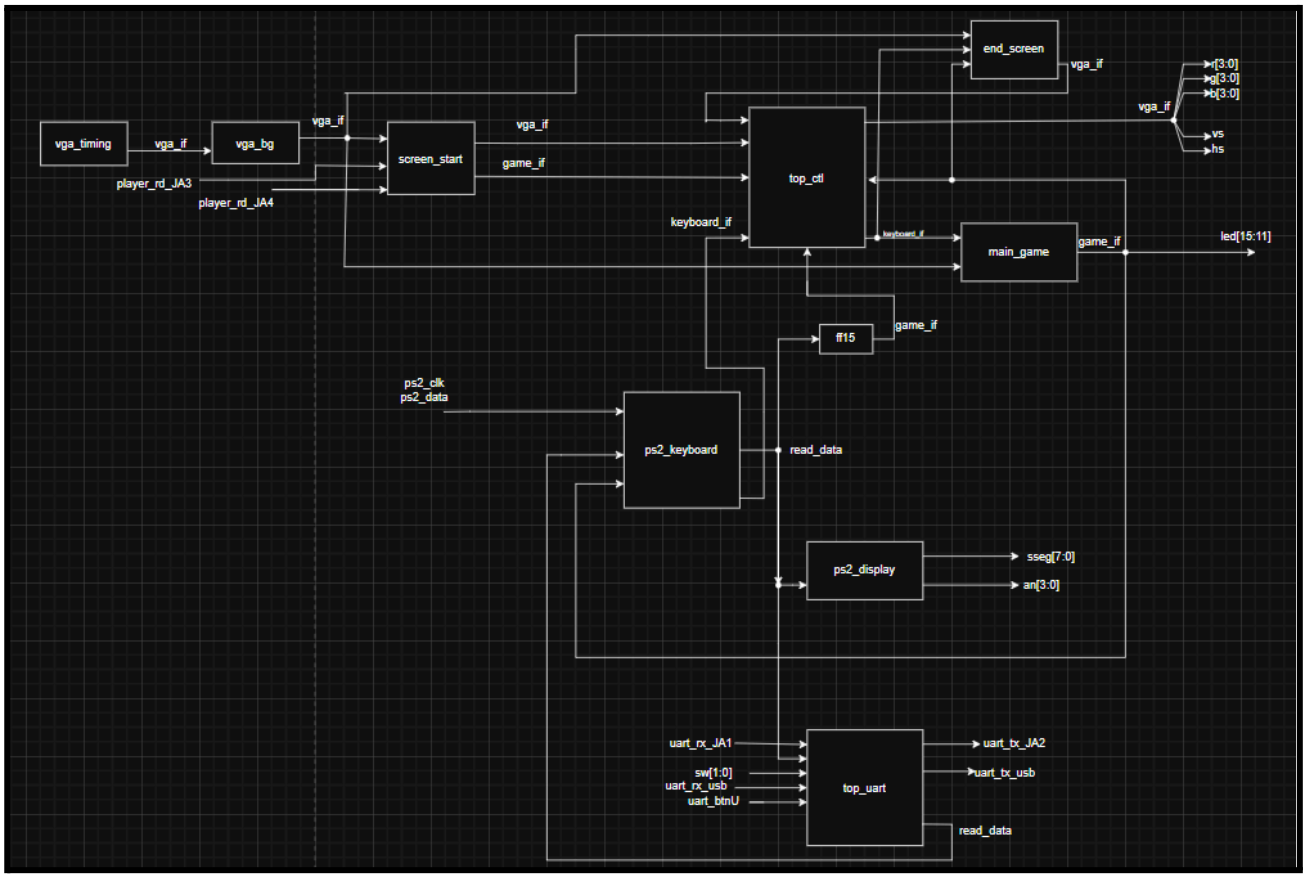
Uwaga: dobrze zrobiony projekt zawiera tylko moduły strukturalne (zbudowane z innych modułów) i funkcjonalne (zawierające bloki proceduralne always @). Staramy się nie generować bloków mieszających te dwa typy, o ile to możliwe (głównym kryterium powinna być czytelność kodu).

Uwaga: opisujemy architekturę **tylko głównego modułu oraz rozprowadzenie sygnału zegara**.

4.1. Moduł: top

Osoba odpowiedzialna: AZ & KW

4.1.1. Schemat blokowy



schemat blokowy modułu głównego

4.1.2. Porty

a) ps2 – ps2_keyboard - input, uart - top_uart - input, sw[1:0] - uart_top - input

nazwa portu	opis
ps2_clk	zegar dla modułu ps2 (inny niż globalne clk)
ps2_data	szeregowe wejście danych
uart_rx_JA1	uart
uart_btnU	zwiększanie o 3 wysyłanego sygnału(tak jak ppkt. lab6, do celów debugowania)
player_rd_JA3	gotowość gracza 1

player_rd_JA4	gotowość gracza 2
sw[2]	zmiana uarta na sygnał z jednej klawiatury (zasadniczo sprawia że drugi basys jest niepotrzebny, używane do ulepszania gry bez konieczności posiadania 2 basysów)
sw[1]	Zmiana między wysyłaniem danych Tx między UART-USB a linią JA2 (domyślnie sw wyłączony -> przesyłaj na lini Tx)
sw[0]	UART odsyła na wyjście Tx to co dostaje na linii Rx (pozostałość z lab6, przydatne do debugowania)

b) vga – top_ctl, output

nazwa portu	opis
vs	sygnał synchronizacji pionowej VGA
hs	sygnał synchronizacji poziomej VGA
r[3:0]	sygnał koloru czerwonego
g[3:0]	sygnał koloru zielonego
b[3:0]	sygnał koloru niebieskiego
sseg[7:0]	pokazywanie kodu przychodzących klawiszy,używane przy debugowaniu (fizycznie 8 ledów tworzących 1 cyfre wyświetlacza 7-mio segmentowego)
an[3:0]	pokazywanie kodu przychodzących klawiszy,używane przy debugowaniu, (na 1 an przypada sseg[7:0]) (fizycznie wyświetlacz 7-mio segmentowy)
led[15:11]	informacja dla developera, używane przy debugowaniu {fire_active, moving [1:0], change_angle [1:0]}

4.1.3. Interfejsy

a) vga_if– vga_timing,bg,start_screen,main_game,end_screen,top_ctl

nazwa sygnału	opis
hcount	horyzontalna pozycja kursora myszy na ekranie
vcount	wertykalna pozycja kursora myszy na ekranie
vblnk	pionowe wygaszenie ekranu
hblnk	poziome wygaszenie ekranu
rgb[11:0]	kolorki
vsync	synchronizacja pionowa
hsync	synchronizacja pozioma

b) ***game_if - ff_15,main_game,end_screen,top_if***

nazwa sygnału	opis
player_turn	sygnał tury danego gracza, blokuje możliwość akcji poza swoją turą
game_over	sygnał informujący o końcu gry wysyłany po wyzerowaniu zdrowia swojego lub przeciwnika
fire_active	pionowe wygaszenie ekranu
moving	sygnał wysyłający informacje o poruszaniu się postaci 1 lub 2 gracza
restart_game	reset

show_help	pokazanie opisu akcji możliwych do podjęcia oraz ogólnych zasad gry
game_over_CODE	sygnał informujący o końcu gry po wpisaniu specjalnego kodu
start_game	sygnał pozwalający uruchomić grę

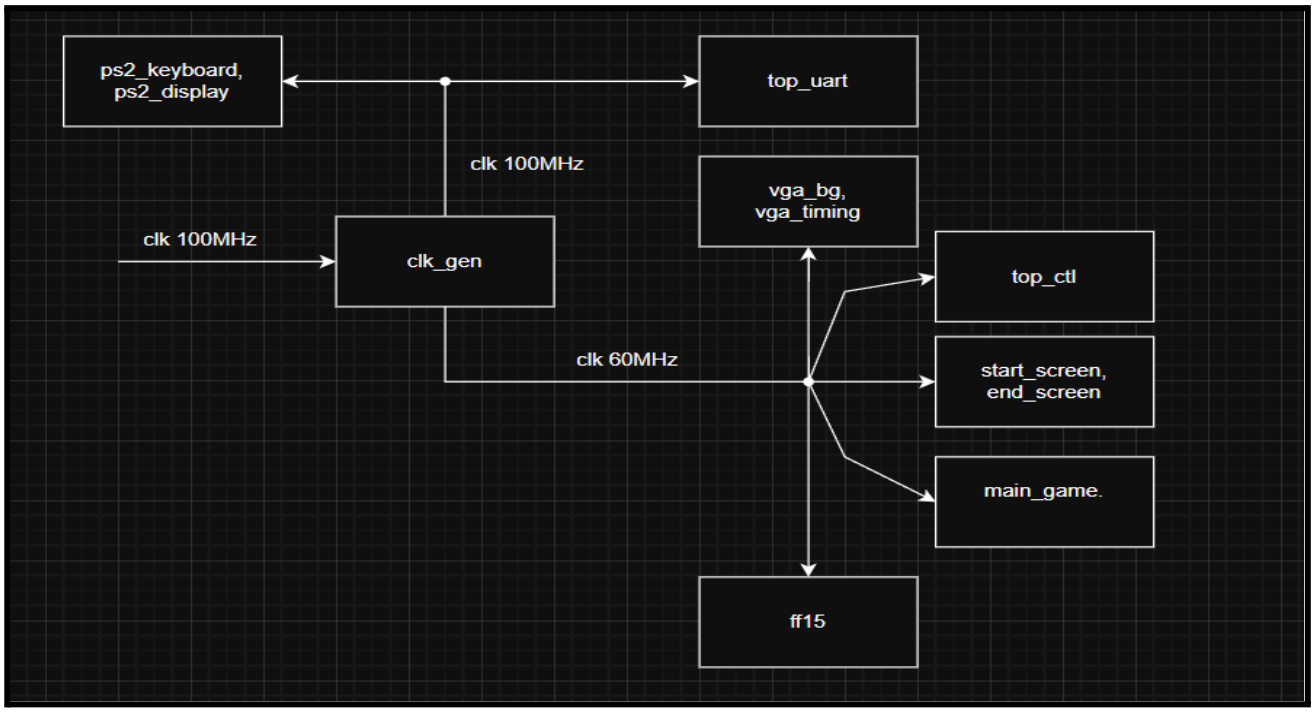
4.2. Rozprowadzenie sygnału zegara

Osoba odpowiedzialna: AZ & KW

Informacja na temat źródła sygnału zegarowego, używanych częstotliwości zegara w całym układzie.

Moduł generatora zegara umieszczamy w module głównym projektu. W pozostałych modułach używamy tylko i wyłącznie sygnały zegara wygenerowane przez ten moduł.

Wygenerowany w IP_Vivado moduł clk_wiz_1 (na schemacie clk_gen) jest umieszczony w pliku fpga/rtl/top_vga_basys3.sv. Oprócz clocków 60 i 100 MHz jest też generowany clk 40 MHz, który ostatecznie nie jest wykorzystywany. Jest on nadal w projekcie ponieważ projekt zaczynał w 800x600 do którego był potrzebny clk 40MHz.
Wykorzystano clocki 60MHz i 100MHz, nie są one niestety wielokrotnościami siebie przez co w projekcie musieliśmy kodować zgodnie z regułami CDC - stąd pliki konieczne do synchronizacji sygnałów, głownie z clk100MHz -> clk60MHz, czyli sygnały pochodzące z ps2 lub uart odpowiednio zsynchronizować do clk60MHz (większość jest robiona w ps2_keyboard_2xff - dla sygnałów 1bitowych wystarczy przeprowadzenie 2 flip-flopy, dla sygnałów wielobitowych jest to bardziej skomplikowane, stąd moduł w delay/sync_handshake (stosowany wyłącznie w FF15 żeby zsynchronizować rx_data z clk60MHz, po to aby go odczytywać i wystawiać game_over_CODE w domenie tego clocka)).



5. Implementacja

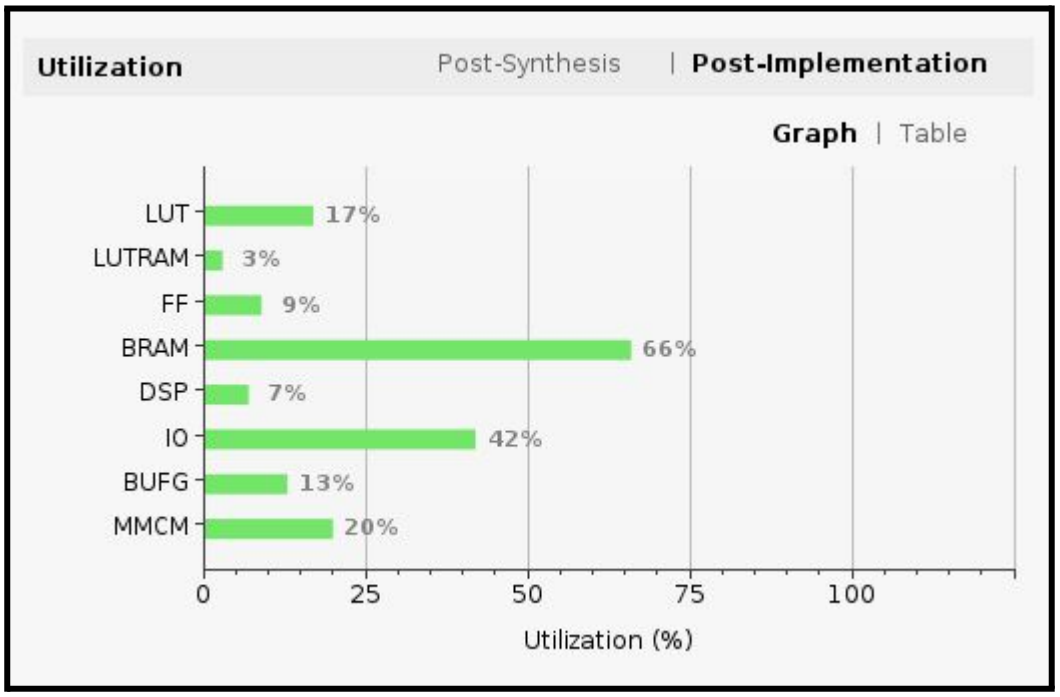
5.1. Lista zignorowanych ostrzeżeń Vivado.

Jedyne dwa mnożenia jakie wykonujemy w całym projekcie są w modułach związanych z rysowaniem interfejsu gracza w trakcie gry i są to obliczenia bar_width_temp dla siły i paliwa.

Identyfikator ostrzeżenia	Liczba wystąpień	Uzasadnienie
[DRC-DPIP-1]	6	Pipeline inputów dla wszystkich sygnałów związanych z mnożeniem mógłby poprawić performance, ale według nas nie wpłynęłoby na tyle znacząco. Total on-chip power nie przekraczał typowych parametrów dla projektu tych rozmiarów. Mnożenie jest robione w modułach draw, dlatego nie chcieliśmy wstawiać dodatkowych opóźnień, żeby nie rozsynchronizować całego projektu (między main_game a start/end_screen)
[DRC-DPOP-1]	6	Pipeline inputów dla wszystkich sygnałów związanych z mnożeniem mógłby poprawić performance, ale według nas nie wpłynęłoby na tyle znacząco. Total on-chip power nie przekraczał typowych parametrów dla projektu tych rozmiarów. Mnożenie jest robione w modułach draw, dlatego nie chcieliśmy wstawiać dodatkowych opóźnień, żeby nie rozsynchronizować całego projektu (między main_game a start/end_screen)
[DRC-DPOP-2]	6	Pipeline inputów dla wszystkich sygnałów związanych z mnożeniem mógłby poprawić performance, ale według nas nie wpłynęłoby na tyle znacząco. Total on-chip power nie przekraczał typowych parametrów dla projektu tych rozmiarów. Mnożenie jest robione w modułach draw, dlatego nie chcieliśmy wstawiać dodatkowych opóźnień, żeby nie rozsynchronizować całego projektu (między main_game a start/end_screen)
[DRC-PDRC-153]	9	Warning dotyczy terrain_y_out, które jest “reliktem” z próby implementacji destrukcji terenu (opisane w pkt. 8). Samo wyjście nie jest w żaden sposób wykorzystywane w projekcie a usunięcie wszystkich sygnałów powiązanych z destrukcją terenu zajęło by zbyt dużą ilość czasu (próbowaliśmy ją zaimplementować aż do samego końca).

5.2. Wykorzystanie zasobów

Tabela z wykorzystaniem zasobów z Vivado



5.3. Marginesy czasowe

Marginesy czasowe (WNS) dla setup i hold.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.151 ns	Worst Hold Slack (WHS):	0.028 ns	Worst Pulse Width Slack (WPWS):	3.000 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	8585	Total Number of Endpoints:	8585	Total Number of Endpoints:	4315

All user specified timing constraints are met.

6. Konfiguracja sprzętu

Osoba odpowiedzialna: AZ & KW

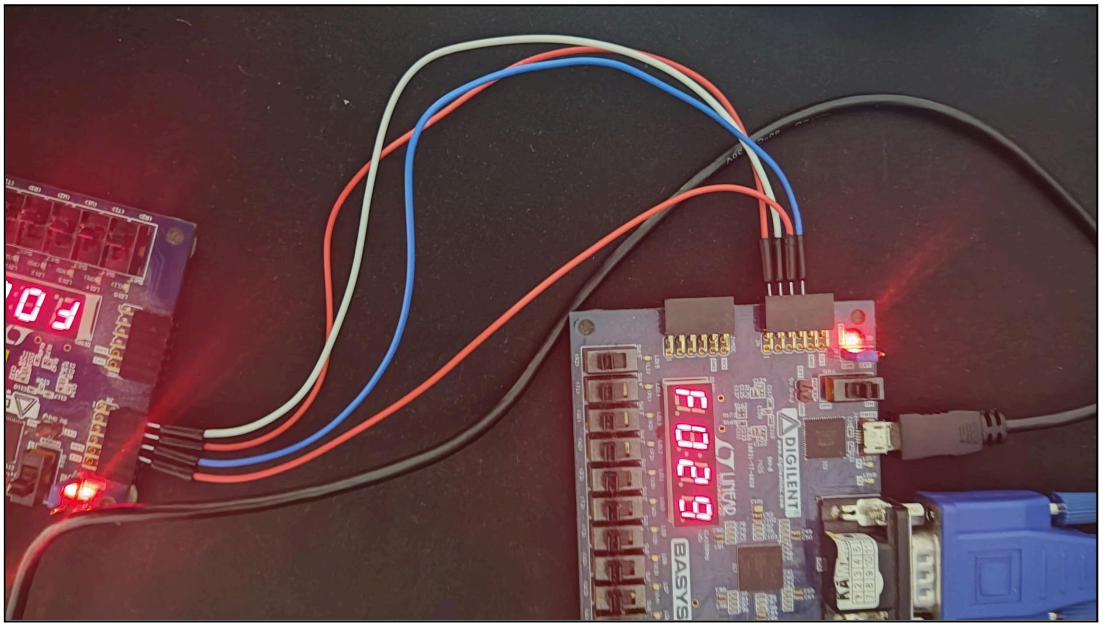
Schemat połączenia ze sobą płytek Basys3 w trybie multiplayer:

Połączenie UARTa:

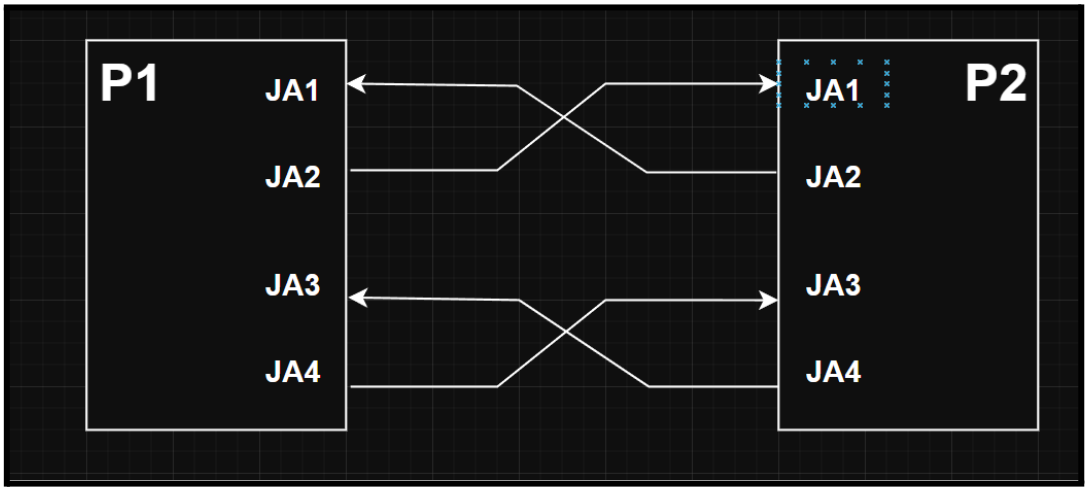
Linia Rx na obu płytkach jest na pinie JA1 a linia Tx na pinie JA2. Należy podłączyć JA1 pierwszego Basysa z JA2 drugiego Basysa. Drugi basys powinien zostać podłączony analogicznie (JA1 z JA2).

Połączenie sygnału ready:

Podobnie jak UART - sygnał ready posiada jedno z wejść (JA3), które służy do odczytu oraz drugie (JA4) do przesyłania sygnału ready. Należy podłączyć port JA3 pierwszego Basysa z portem JA4 drugiego Basysa.



Przykładowe podłączenie sygnałów UART i Player_ready



Uproszczony schemat

!! WAŻNE !!

Klawiatura ps2 używa kodów scancode make/break. Maszyna stanów dekodująca te wartości jest w ps2_keyboard/ps2_keyboard_ctl.sv. Tam można zaobserwować jaki kod odpowiada jakiemu klawiszowi - i większość klawiszy jest zgodna ze scancode's set 2 - oprócz strzałek w górę, dół i lewo - powinno je poprzedzać E0 (czyli e0_flag powinno być 1), ale klawatura, której używaliśmy do debugu - a zarazem jedyna jaką mieliśmy która działała z ps2 - nie poprzedzała tych trzech strzałek kodem E0. Jeżeli klawiatura będzie wysyłać E0 przed tymi klawiszami (łatwo to sprawdzić - mając wgrany bitstream wystarczy wciskać dane klawisze - ich kod pokaże się na hex_display, najlepiej przytrzymać), trzeba zmienić wartości dla danych case'ów i niestety wygenerować bitstream jeszcze raz - trwa to ok 5-10min na 1 gracza (zmieniając graczy trzeba zmienić parametr PLAYER_NUMBER w top_vga.sv, = 1 to PLAYER1, =2 PLAYER2 oraz nazwę wygenerowanego bitstreamu, żeby nie nadpisał poprzedniego).

W folderze results są dwa bitstreamy, każdy dla jednego gracza. Zgodnie z nazwami PLAYER1 będzie graczem pierwszym, a PLAYER2 graczem drugim. Należy wgrać jeden bitstream na jedną płytkę, drugi na drugą. Rozdzielność bitstreamów ma tu znaczenie tylko przy synchronizacji tur, tak, aby docelowo oba basysy mogły wykonywały to samo, to znaczy - mają wyświetlać ten sam stan gry dając w rezultacie tą samą rozgrywaną partię na obu ekranach (bez przesyłania miliona danych, związanych

z pozycjami czołgu, pocisku, etc). Niestety mieliśmy dostępny tylko 1 monitor VGA i działanie tego nie zostało w pełni przetestowane. Gra powinna działać na dwóch osobnych ekranach (działania gracza powinny się aktualizować na dla PLAYER1 i PLAYER2, co na pewno wiemy, że nie zadziała poprawnie to restart synchroniczny, wewnętrzny(internal_rst) gry; pod koniec gry internal rst działał poprawnie w wersjach przed synchronizacją urządzeń (wtedy tryb gry był: jeden Basys (Player1) podpięty do monitora i klawiatury, drugi służy jako klawiatura dla gracza 2). PLAYER1 jest docelowym bitstreamem, wokół którego cały projekt był tworzony. PLAYER2 może równie dobrze działać jako sam kontroler, który na swoje wyjście JA2 (linia tx uarta) wysyła na linie rx PLAYER1 (JA1) input z klawiatury. Aby program tak zadziałał, należy na obu basysach włączyć sw[2] i kliknąć enter, następnie wyłączyć switche).

Aby rozpocząć grę, oba Basysy muszą wcisnąć enter chociaż raz. Po zakończonej partii albo FF15 należy zresetować oba urządzenia za pomocą btnC.

7. Film.

Link do ściągnięcia filmu:

https://

8. Niezaimplementowane funkcje.

w programie jest trochę funkcji, których nie udało nam się dopracować do takiego stopnia, że bylibyśmy zadowoleni i mogli zostawić w ostatecznej wersji. Tutaj wymienimy te funkcje.

- **destrukcja terenu**: nie udało nam się w pełni dopracować efektu na tyle że chcielibyśmy go zaprezentować. Sygnały w module main_game z tym związane to głównie wszystkie terrain_y i write_en, które miały za zadanie wpisywać nowe wartości terenu do terrain_ram (który w obecnej wersji działa obecnie jak zwykły rom ze względu na niedopracowanie tego punktu, po prostu ustawiliśmy write_en stale na 0).
- **reset wewnętrzny synchroniczny**: działał poprawnie wyłącznie w trybie OneBasysMode (sw[2] == 1) przed zaimplementowaniem synchronizacji. Nie doszliśmy jeszcze do przyczyny tego problemu ale podejrzewamy problemy w top_vga_ctl. **Po zakończonej partii albo FF15 należy zresetować oba urządzenia za pomocą btnC.**
- **tank movement testbench:** jest to testbench, który wyorzystywaliśmy na początku do sprawdzania mechaniki poruszania się, gdy byliśmy pewni że działa, nie aktualizowaliśmy tego testbencha.
- UART i synchronizacja były robione jako ostatnie i dość późno, dlatego dla przyśpieszenia procesu nie zastosowaliśmy się w pełni do uwagi do architektury - w top_vga znajduje się jeden always_comb, dzięki któremu każdy gracz wysyła sygnał ready po liniach JA3, JA4. Zdajemy sobię sprawę, że powinniśmy to zamknąć w top_vga_ctl lub w osobnym module, do momentu synchronizacji wszystkie moduły były robione zgodnie z tą uwagą.
- Był koncept zmiany rozmiarów czołgów do 32x32, ponieważ w obecnej wersji gra jest trochę zbyt prosta, ale wymagałoby to o wiele większej ilości zmian niż tylko zmiana zdjęcia (osobne zdjęcia lufy, aktualizacja barrel_ctl)

Było wiele pomysłów i wiele prób, ale ostatecznie pokonał nas czas.